The MQ-Calculus in Lean 4: Communication-as-Measurement, Operational and Denotational Semantics

Zar Oruži*

March 1, 2026

Abstract

We present a Lean 4 formalization of the MQ-calculus of Stay and Meredith [1], comprising 1,413 lines across 10 files with zero sorries and zero custom axioms. The MQ-calculus unifies communication and quantum measurement: a COMM event produces a binary measurement outcome governed by Born-rule probabilities.

The formalization covers: (1) process syntax with de Bruijn wire indices; (2) the COMM reduction rule with constructive both-outcomes witness; (3) structural congruence (parallel laws, scope extrusion via shift); (4) full operational reduction with contextual closure; (5) a backend-parametric denotational semantics with a statevector instantiation including post-collapse normalization; and (6) clause-level theorem handles (`paper63_*`) for paper auditing.

Cross-calculus bridges connect MQ measurement branching to MORK binary-fold outcomes and place the calculus in the broader OSLF type synthesis pipeline.

1 Why MQ-Calculus Matters

The calculus serves three simultaneous roles.

1. It stress-tests the MeTTaIL formal stack on a nontrivial stochastic process calculus.
2. It provides concrete benchmarks for stochastic simulators beyond the stochastic π-calculus setting.
3. It grounds the OSLF program in physically meaningful scenarios where observations (experiments) are represented structurally in the reduction semantics.

A central methodological point is to represent interpretational choices as program structure, not as external metaphysical commitments. Distinct quantum narratives correspond to distinct valid programs with distinct typing/behavioral footprints.

2 Paper-to-Lean Map

The Lean formalization is organized under `Mettapedia/Languages/ProcessCalculi/MQCalculus/`:

*"Oruži" denotes AI-assisted collaboration using Claude Code (Anthropic), ChatGPT, and Codex (OpenAI). Formalization of the MQ-calculus by Stay and Meredith [1].

File	Contents	Lines
`Syntax.lean`	Process grammar, gate specs	120
`CommRule.lean`	COMM rule, measurement branches	95
`Reduction.lean`	Operational reduction, multi-step	172
`StructuralCongruence.lean`	SC: par laws, scope extrusion	205
`Shift.lean`	De Bruijn wire shifting	110
`Backend.lean`	QState, Born probs, collapse, backend	248
`Denotational.lean`	`denoteWith`, weighted branches	165
`PaperMap.lean`	`paper63_*` theorem handles	130
`Interoperability.lean`	MORK binary-fold bridge	78
`MQCalculus.lean`	Barrel import	10
Total	**0 sorries, 0 axioms**	**1,413**

3 Core Syntax in Lean

The process grammar is encoded with de Bruijn-style wire indexing and typed gate specifications.

Listing 1: Process and gate syntax (from `Syntax.lean`)

```
inductive GateOp : Type where
  | H
  | X
  | Z
  | custom (name : String)
  deriving DecidableEq, Repr

structure GateSpec where
  op : GateOp
  target : Nat
  deriving DecidableEq, Repr

def gateSpecOfString : String -> GateSpec := GateSpec.ofString

inductive Process : Type where
  | MQNil  : Process
  | MQPar  : Process -> Process -> Process
  | MQNu   : Process -> Process
  | MQGate : GateSpec -> Process -> Process
  | MQOut  : Nat -> Process
  | MQIn   : Nat -> Process -> Process -> Process
  deriving DecidableEq, Repr
```

This representation keeps the surface small while supporting typed gate routing and explicit branch continuations for measurement outcomes.

4 Communication as Measurement

The communication rule is represented by a branch type plus a small-step relation that produces both outcome branches constructively.

Listing 2: COMM branching object and witnesses (from `CommRule.lean`)

```
inductive Outcome : Type where
```

```
  | zero : Outcome
  | one  : Outcome
  deriving DecidableEq

structure MeasurementBranch where
  outcome : Outcome
  result  : Process

inductive CommReduction : Nat -> Process -> Process -> MeasurementBranch -> Prop where
  | outcome_zero (i : Nat) (p q : Process) :
      CommReduction i p q (.zero, p)
  | outcome_one (i : Nat) (p q : Process) :
      CommReduction i p q (.one, q)

theorem comm_both_outcomes (i : Nat) (p q : Process) :
    CommReduction i p q (.zero, p) /\ CommReduction i p q (.one, q) :=
  (.outcome_zero i p q, .outcome_one i p q)
```

Born-style probabilities are defined separately on vectors. The branch relation itself remains structural and proof-friendly.

5 Operational Reduction Relation

Full one-step reduction includes COMM firing, structural congruence closure, and contextual closure under parallel/new/gate constructors.

Listing 3: One-step and multi-step reduction (from `Reduction.lean`)

```
inductive Reduces : Process -> Process -> Prop where
  | comm (i : Nat) (p q : Process) (b : MeasurementBranch) :
      CommReduction i p q b ->
      Reduces (MQPar (MQOut i) (MQIn i p q)) b.result
  | sc (p p' q' q : Process) :
      SC p p' -> Reduces p' q' -> SC q' q -> Reduces p q
  | par_l  (p p' q : Process) : Reduces p p' -> Reduces (MQPar p q) (MQPar p' q)
  | par_r  (p q q' : Process) : Reduces q q' -> Reduces (MQPar p q) (MQPar p q')
  | nu_step  (p p' : Process) : Reduces p p' -> Reduces (MQNu p) (MQNu p')
  | gate_step (g : GateSpec) (p p' : Process) :
      Reduces p p' -> Reduces (MQGate g p) (MQGate g p')

inductive MultiStep : Process -> Process -> Prop where
  | refl (p : Process) : MultiStep p p
  | step (p q r : Process) : Reduces p q -> MultiStep q r -> MultiStep p r
```

6 Backend-Parametric Denotational Semantics

The semantics is parameterized by a backend interface with explicit operations for gate action, fresh allocation, branch probabilities, and collapse.

Listing 4: Runtime state and backend interface (from `Backend.lean`)

```
structure QState where
  n : Nat
```

```
  psi : QVec n

def rawOutcomeProb (i : Nat) (out : Outcome) (st : QState) : Real :=
  Finset.univ.sum (fun k : Fin (2 ^ st.n) =>
    if bitMatches i k.1 out then Complex.normSq (st.psi.ofLp k) else 0)

def collapseByOutcome (i : Nat) (out : Outcome) (st : QState) : QState :=
  let p := rawOutcomeProb i out st
  if p = 0 then st else
    let z : Complex := ((Real.sqrt p) : Complex)
    let psi' : QVec st.n :=
      Finset.univ.sum (fun k : Fin (2 ^ st.n) =>
        EuclideanSpace.single k
          (if bitMatches i k.1 out then st.psi.ofLp k / z else 0))
    (st.n, psi')

structure MQSemanticsBackend where
  branchProb : Nat -> Outcome -> QState -> Real
  branchProb_nonneg : forall i out st, 0 <= branchProb i out st
  branchProb_sum_one : forall i st,
    branchProb i .zero st + branchProb i .one st = 1
  applyGate : GateSpec -> QState -> QState
  allocFresh : QState -> QState
  collapse : Nat -> Outcome -> QState -> QState
```

The current `statevectorBackend` instantiation gives executable semantics while preserving theorem-level invariants such as nonnegativity, normalization splits, and collapse behavior.

7 Executable Denotation and Measurement Selection

The denotation function is structurally recursive on process syntax and resolves `MQIn` by comparing branch probabilities.

Listing 5: Denotational clauses and weighted measurement view (from `Denotational.lean`)

```
def denoteWith (backend : MQSemanticsBackend) : Process -> QState -> Option QState
  | .MQNil,      st => some st
  | .MQOut _,    st => some st
  | .MQGate s p, st => denoteWith backend p (backend.applyGate s st)
  | .MQNu p,     st => denoteWith backend p (backend.allocFresh st)
  | .MQPar p q,  st => (denoteWith backend p st).bind (denoteWith backend q)
  | .MQIn i p q, st =>
      if backend.branchProb i .zero st >= backend.branchProb i .one st then
        denoteWith backend p (backend.collapse i .zero st)
      else
        denoteWith backend q (backend.collapse i .one st)

structure WeightedBranch where
  prob_zero    : Real
  prob_one     : Real
  state_zero   : QState
  state_one    : QState
  prob_zero_nn : 0 <= prob_zero
  prob_one_nn  : 0 <= prob_one
```

```
  prob_sum_le  : prob_zero + prob_one <= 1
```

This gives two aligned views: deterministic branch selection for executable denotation and explicit two-branch packaging for proof/audit work.

8 Clause-Level Theorem Map

The theorem index module makes paper auditing straightforward.

Listing 6: Paper-clause theorem handles (from `PaperMap.lean`)

```
theorem paper63_nil ...
theorem paper63_out ...
theorem paper63_gate ...
theorem paper63_new ...
theorem paper63_par ...
theorem paper63_in ...
theorem paper63_measurement_probs_sum_one ...
theorem paper63_measurement_state_zero ...
theorem paper63_measurement_state_one ...
theorem paper63_statevector_collapse_norm_one_of_raw_pos ...
theorem paper63_comm_denote_eq_measurement_selection ...
```

A key normalization result used in review is:

Listing 7: Post-collapse normalization (from `Backend.lean`)

```
theorem collapseByOutcome_norm_eq_one_of_raw_pos
    (i : Nat) (out : Outcome) (st : QState)
    (hraw : 0 < rawOutcomeProb i out st) :
    norm ((collapseByOutcome i out st).psi) = 1
```

9 Cross-Calculus Bridges

9.1 MQ and MORK

MQ measurement branching is linked to MORK binary-fold branching via an explicit bridge theorem:

Listing 8: MQ–MORK bridge (from `Interoperability.lean`)

```
theorem comm_nondeterminism_iff_mork_binary (i : Nat) (p q : Process) :
    (CommReduction i p q (.zero, p) /\ CommReduction i p q (.one, q)) <->
    forall (fold : FoldStep) (_hb : fold.isBinary),
      (exists (f0 : FoldStep) (_ : f0.isBinary),
        FoldPicksSubResult f0 hbin .subResult0) /\
      (exists (f1 : FoldStep) (_ : f1.isBinary),
        FoldPicksSubResult f1 hbin .subResult1)
```

This ties communication-level nondeterminism in MQ directly into the MORK three-phase execution model.

9.2 Shared Process Calculus Infrastructure

The Mettapedia formalization contains three process calculi—π-calculus (8,768 lines), ρ-calculus (4,848 lines), and MQ-calculus—that share ∼40% of their reduction/congruence infrastructure. A shared library in `ProcessCalculi/Common/` eliminates this duplication:

File	Contents	Lines
`Common/Star.lean`	`RTClosure` (Type) + `RTClosureProp` (Prop) closures Generic lifters: `liftUnary`, `liftBinLeft`, `liftBinRight`	140
`Common/Congruence.lean`	`HasSC` typeclass, `reduces_modulo_sc`	65
`Common/ProcessAlgebra.lean`	`HasPar`, `HasNu`, `HasNil` typeclasses	30

Each calculus registers instances:

- **π-calculus**: `HasPar`, `HasNil`, `HasSC` (wrapping Type-valued SC in `Nonempty`).
- **ρ-calculus**: `HasSC` (Prop-valued SC directly). The ρ-calculus also *gained* multi-step parallel congruence lifting (`ReducesStar.par_head`, `ReducesStar.par_any_pos`) that was previously missing.
- **MQ-calculus**: `HasPar`, `HasNil`, `HasNu`, `HasSC`.

All three calculi share:

- **Structural congruence**: equivalence relation with parallel commutativity/associativity and scope extrusion, unified via `HasSC`.
- **COMM rule shape**: sender + receiver → result, differing only in payload consumption (substitution in π/ρ, measurement in MQ).

9.3 Spice Calculus and Present Moment

The Spice calculus (formalized in `RhoCalculus/SpiceRule.lean`) extends ρ-calculus COMM with n-step lookahead: agents receive the set of states reachable in $\leq n$ steps before committing to an interaction. When $n = 0$ this recovers standard ρ-calculus COMM; the "present moment" ($n = 1$) gives agents awareness of their immediate interaction possibilities.

The new `ReducesN.par_head` and `ReducesN.par_any_pos` theorems (n-step parallel congruence lifting) enable compositional reasoning about Spice agents: an agent's future states lift cleanly through parallel contexts (`futureStates_par_head`, `presentMoment_par_head`, `reachableStates_par_head`). This is essential for the Spice rule, where agents must evaluate their environment's n-step evolution.

10 Wigner-Style Program Distinctions

A practical interpretational takeaway is representational: one can encode a fully unitary evolution program and, separately, a staged-communication program where measurement-triggering communications occur between stages. In this framework they are different programs with different transition structures. The formal machinery does not force them to collapse into a single execution narrative.

11 OSLF Context and AGI-Relevant Direction

The Leifer–Milner–Sewell context-generation perspective provides the right conceptual anchor: contexts act as experiments. In MQ settings this dovetails with communication-as-measurement and gives a principled route from rewrite semantics to observational structure.

For AGI-motivated scientific discovery workflows, the important systems consequence is that physical theories can be encoded as rewrite theories, transformed, and evaluated in a semantics-aware loop. Extending from MQ-calculus to additional physics rewrite systems (e.g., GR-oriented systems) becomes a matter of language-instance construction and bridge theorems, not ad hoc simulator rewrites.

12 Next Formalization Targets

1. **Multi-qubit gate semantics** beyond single-wire gate action.
2. **Stronger collapse theory**: preservation under composition, idempotence of repeated measurement on the same wire.
3. **OSLF integration**: MQ process experiments as OSLF language instances with $\Diamond \dashv \Box$ Galois connection, paralleling the PeTTa and Governance OSLF instances.
4. **Spice–MQ bridge**: connect the ρ-calculus Spice n-step lookahead to MQ measurement prediction (agents observing future measurement outcomes).

References

[1] M. Stay and L. G. Meredith, *The MQ-Calculus: A Calculus for Quantum Processes Involving Intermittent Measurement*, 2026 working draft.

[2] L. G. Meredith, *How the Agents Got Their Present Moment*, 2026.

[3] L. G. Meredith and M. Radestock, *A Reflective Higher-Order Calculus*, *Electronic Notes in Theoretical Computer Science*, 141(5), 2005.

[4] S. J. Gay and R. Nagarajan, *Communicating Quantum Processes*, POPL, 2005.

[5] J. Leifer and R. Milner, *Deriving Bisimulation Congruences for Reactive Systems*, CONCUR, 2000.